RPG Sheet Builder V1 - Codigo-fonte para entrega

Conteudo incluido:
- Configuracao do projeto Flutter
- Codigo-fonte em lib/
- Dados locais em assets/data/
- Testes automatizados em test/

Arquivos gerados, build/ e executaveis nao foram incluidos.
==

==
ARQUIVO: pubspec.yaml
==
```
name: rpg_sheet_builder
description: "Versao de teste do RPG Sheet Builder com motor de regras reativo."
# The following line prevents the package from being accidentally published to
# pub.dev using `flutter pub publish`. This is preferred for private packages.
publish_to: 'none' # Remove this line if you wish to publish to pub.dev

# The following defines the version and build number for your application.
# A version number is three numbers separated by dots, like 1.2.43
# followed by an optional build number separated by a +.
# Both the version and the builder number may be overridden in flutter
# build by specifying --build-name and --build-number, respectively.
# In Android, build-name is used as versionName while build-number used as versionCode.
# Read more about Android versioning at https://developer.android.com/studio/publish/versioning
# In iOS, build-name is used as CFBundleShortVersionString while build-number is used as
CFBundleVersion.
# Read more about iOS versioning at
# https://developer.apple.com/library/archive/documentation/General/Reference/InfoPlistKeyReference/Arti
cles/CoreFoundationKeys.html
# In Windows, build-name is used as the major, minor, and patch parts
# of the product and file versions while build-number is used as the build suffix.
version: 1.0.0+1

environment:
  sdk: ^3.12.0

# Dependencies specify other packages that your package needs in order to work.
# To automatically upgrade your package dependencies to the latest versions
# consider running `flutter pub upgrade --major-versions`. Alternatively,
# dependencies can be manually updated by changing the version numbers below to
# the latest version available on pub.dev. To see which dependencies have newer
# versions available, run `flutter pub outdated`.
dependencies:
  flutter:
    sdk: flutter

  # The following adds the Cupertino Icons font to your application.
  # Use with the CupertinoIcons class for iOS style icons.
  cupertino_icons: ^1.0.8
  flutter_riverpod: ^3.3.1
  go_router: ^17.2.3

dev_dependencies:
  flutter_test:
    sdk: flutter

  # The "flutter_lints" package below contains a set of recommended lints to
  # encourage good coding practices. The lint set provided by the package is
  # activated in the `analysis_options.yaml` file located at the root of your
  # package. See that file for information about deactivating specific lint
  # rules and activating additional ones.
  flutter_lints: ^6.0.0

# For information on the generic Dart part of this file, see the
# following page: https://dart.dev/tools/pub/pubspec

# The following section is specific to Flutter packages.
flutter:

  # The following line ensures that the Material Icons font is
  # included with your application, so that you can use the icons in
  # the material Icons class.
```

```
  uses-material-design: true

  assets:
    - assets/data/

  # An image asset can refer to one or more resolution-specific "variants", see
  # https://flutter.dev/to/resolution-aware-images

  # For details regarding adding assets from package dependencies, see
  # https://flutter.dev/to/asset-from-package

  # To add custom fonts to your application, add a fonts section here,
  # in this "flutter" section. Each entry in this list should have a
  # "family" key with the font family name, and a "fonts" key with a
  # list giving the asset and other descriptors for the font. For
  # example:
  # fonts:
  #   - family: Schyler
  #     fonts:
  #       - asset: fonts/Schyler-Regular.ttf
  #       - asset: fonts/Schyler-Italic.ttf
  #         style: italic
  #   - family: Trajan Pro
  #     fonts:
  #       - asset: fonts/TrajanPro.ttf
  #       - asset: fonts/TrajanPro_Bold.ttf
  #         weight: 700
  #
  # For details regarding fonts from package dependencies,
  # see https://flutter.dev/to/font-from-package
```

```
================================================================================================================
ARQUIVO: analysis_options.yaml
================================================================================================================
# This file configures the analyzer, which statically analyzes Dart code to
# check for errors, warnings, and lints.
#
# The issues identified by the analyzer are surfaced in the UI of Dart-enabled
# IDEs (https://dart.dev/tools#ides-and-editors). The analyzer can also be
# invoked from the command line by running `flutter analyze`.

# The following line activates a set of recommended lints for Flutter apps,
# packages, and plugins designed to encourage good coding practices.
include: package:flutter_lints/flutter.yaml

linter:
  # The lint rules applied to this project can be customized in the
  # section below to disable rules from the `package:flutter_lints/flutter.yaml`
  # included above or to enable additional rules. A list of all available lints
  # and their documentation is published at https://dart.dev/lints.
  #
  # Instead of disabling a lint rule for the entire project in the
  # section below, it can also be suppressed for a single line of code
  # or a specific dart file by using the `// ignore: name_of_lint` and
  # `// ignore_for_file: name_of_lint` syntax on the line or in the file
  # producing the lint.
  rules:
    # avoid_print: false  # Uncomment to disable the `avoid_print` rule
    # prefer_single_quotes: true  # Uncomment to enable the `prefer_single_quotes` rule

# Additional information about this file can be found at
# https://dart.dev/guides/language/analysis-options
```

```
================================================================================================================
ARQUIVO: lib/core/constants/attribute_keys.dart
================================================================================================================
const attributeKeys = <String>[
  'strength',
  'dexterity',
  'constitution',
  'intelligence',
  'wisdom',
  'charisma',
];

const attributeLabels = <String, String>{
```

```
  'strength': 'Forca',
  'dexterity': 'Destreza',
  'constitution': 'Constituicao',
  'intelligence': 'Inteligencia',
  'wisdom': 'Sabedoria',
  'charisma': 'Carisma',
};

================================================================================================================
ARQUIVO: lib/core/routing/app_router.dart
================================================================================================================
import 'package:go_router/go_router.dart';

import '../../presentation/screens/character_builder_screen.dart';

final appRouter = GoRouter(
  routes: [
    GoRoute(
      path: '/',
      builder: (context, state) => const CharacterBuilderScreen(),
    ),
  ],
);

================================================================================================================
ARQUIVO: lib/core/theme/app_theme.dart
================================================================================================================
import 'package:flutter/material.dart';

ThemeData buildAppTheme() {
  final colorScheme = ColorScheme.fromSeed(
    seedColor: Colors.indigo,
    brightness: Brightness.light,
  );

  return ThemeData(
    colorScheme: colorScheme,
    useMaterial3: true,
    scaffoldBackgroundColor: colorScheme.surface,
    appBarTheme: AppBarTheme(
      centerTitle: false,
      backgroundColor: colorScheme.primaryContainer,
      foregroundColor: colorScheme.onPrimaryContainer,
    ),
  );
}

================================================================================================================
ARQUIVO: lib/data/datasources/asset_json_datasource.dart
================================================================================================================
import 'dart:convert';

import 'package:flutter/services.dart';

class AssetJsonDatasource {
  AssetJsonDatasource({AssetBundle? bundle}) : _bundle = bundle ?? rootBundle;

  final AssetBundle _bundle;

  Future<List<Map<String, dynamic>>> loadList(String path) async {
    final raw = await _bundle.loadString(path);
    final decoded = jsonDecode(raw) as List<dynamic>;
    return decoded.cast<Map<String, dynamic>>();
  }
}

================================================================================================================
ARQUIVO: lib/data/repositories/asset_game_catalog_repository.dart
================================================================================================================
import '../../domain/entities/background.dart';
import '../../domain/entities/character_class.dart';
import '../../domain/entities/game_catalog.dart';
import '../../domain/entities/proficiency.dart';
import '../../domain/entities/race.dart';
import '../../domain/repositories/game_catalog_repository.dart';
import '../datasources/asset_json_datasource.dart';
```

```
class AssetGameCatalogRepository implements GameCatalogRepository {
  const AssetGameCatalogRepository(this._datasource);

  final AssetJsonDatasource _datasource;

  @override
  Future<GameCatalog> loadCatalog() async {
    final results = await Future.wait([
      _datasource.loadList('assets/data/races.json'),
      _datasource.loadList('assets/data/classes.json'),
      _datasource.loadList('assets/data/backgrounds.json'),
      _datasource.loadList('assets/data/proficiencies.json'),
    ]);

    return GameCatalog(
      races: results[0].map(Race.fromJson).toList(),
      classes: results[1].map(CharacterClass.fromJson).toList(),
      backgrounds: results[2].map(Background.fromJson).toList(),
      proficiencies: results[3].map(Proficiency.fromJson).toList(),
    );
  }
}

================================================================================================
ARQUIVO: lib/domain/entities/attribute_set.dart
================================================================================================
import '../../core/constants/attribute_keys.dart';

class AttributeSet {
  const AttributeSet({
    required this.strength,
    required this.dexterity,
    required this.constitution,
    required this.intelligence,
    required this.wisdom,
    required this.charisma,
  });

  const AttributeSet.standard()
    : strength = 10,
      dexterity = 10,
      constitution = 10,
      intelligence = 10,
      wisdom = 10,
      charisma = 10;

  final int strength;
  final int dexterity;
  final int constitution;
  final int intelligence;
  final int wisdom;
  final int charisma;

  int valueFor(String key) {
    return switch (key) {
      'strength' => strength,
      'dexterity' => dexterity,
      'constitution' => constitution,
      'intelligence' => intelligence,
      'wisdom' => wisdom,
      'charisma' => charisma,
      _ => throw ArgumentError.value(key, 'key', 'Unknown attribute key'),
    };
  }

  AttributeSet setValue(String key, int value) {
    return copyWith(
      strength: key == 'strength' ? value : null,
      dexterity: key == 'dexterity' ? value : null,
      constitution: key == 'constitution' ? value : null,
      intelligence: key == 'intelligence' ? value : null,
      wisdom: key == 'wisdom' ? value : null,
      charisma: key == 'charisma' ? value : null,
    );
  }
```

```
  AttributeSet addBonuses(Map<String, int> bonuses) {
    var result = this;
    for (final entry in bonuses.entries) {
      result = result.setValue(
        entry.key,
        result.valueFor(entry.key) + entry.value,
      );
    }
    return result;
  }

  Map<String, int> toMap() {
    return {for (final key in attributeKeys) key: valueFor(key)};
  }

  factory AttributeSet.fromJson(Map<String, dynamic> json) {
    return AttributeSet(
      strength: json['strength'] as int? ?? 10,
      dexterity: json['dexterity'] as int? ?? 10,
      constitution: json['constitution'] as int? ?? 10,
      intelligence: json['intelligence'] as int? ?? 10,
      wisdom: json['wisdom'] as int? ?? 10,
      charisma: json['charisma'] as int? ?? 10,
    );
  }

  AttributeSet copyWith({
    int? strength,
    int? dexterity,
    int? constitution,
    int? intelligence,
    int? wisdom,
    int? charisma,
  }) {
    return AttributeSet(
      strength: strength ?? this.strength,
      dexterity: dexterity ?? this.dexterity,
      constitution: constitution ?? this.constitution,
      intelligence: intelligence ?? this.intelligence,
      wisdom: wisdom ?? this.wisdom,
      charisma: charisma ?? this.charisma,
    );
  }
}

================================================================================================
ARQUIVO: lib/domain/entities/background.dart
================================================================================================
class Background {
  const Background({
    required this.id,
    required this.name,
    required this.skillProficiencies,
  });

  final String id;
  final String name;
  final List<String> skillProficiencies;

  factory Background.fromJson(Map<String, dynamic> json) {
    return Background(
      id: json['id'] as String,
      name: json['name'] as String,
      skillProficiencies: List<String>.from(
        json['skillProficiencies'] as List? ?? const [],
      ),
    );
  }
}

================================================================================================
ARQUIVO: lib/domain/entities/character.dart
================================================================================================
import 'attribute_set.dart';
```

```
class Character {
  const Character({
    required this.name,
    required this.raceId,
    required this.classId,
    required this.backgroundId,
    required this.level,
    required this.attributes,
  });

  const Character.initial()
    : name = '',
      raceId = 'elf',
      classId = 'fighter',
      backgroundId = 'soldier',
      level = 1,
      attributes = const AttributeSet.standard();

  final String name;
  final String raceId;
  final String classId;
  final String backgroundId;
  final int level;
  final AttributeSet attributes;

  Character copyWith({
    String? name,
    String? raceId,
    String? classId,
    String? backgroundId,
    int? level,
    AttributeSet? attributes,
  }) {
    return Character(
      name: name ?? this.name,
      raceId: raceId ?? this.raceId,
      classId: classId ?? this.classId,
      backgroundId: backgroundId ?? this.backgroundId,
      level: level ?? this.level,
      attributes: attributes ?? this.attributes,
    );
  }
}

================================================================================================
ARQUIVO: lib/domain/entities/character_class.dart
================================================================================================
class CharacterClass {
  const CharacterClass({
    required this.id,
    required this.name,
    required this.hitDice,
    required this.savingThrows,
    required this.proficiencies,
  });

  final String id;
  final String name;
  final int hitDice;
  final List<String> savingThrows;
  final List<String> proficiencies;

  factory CharacterClass.fromJson(Map<String, dynamic> json) {
    return CharacterClass(
      id: json['id'] as String,
      name: json['name'] as String,
      hitDice: json['hitDice'] as int,
      savingThrows: List<String>.from(
        json['savingThrows'] as List? ?? const [],
      ),
      proficiencies: List<String>.from(
        json['proficiencies'] as List? ?? const [],
      ),
    );
  }
}
```

```
================================================================================================================
ARQUIVO: lib/domain/entities/character_stats.dart
================================================================================================================
import 'attribute_set.dart';

class CharacterStats {
  const CharacterStats({
    required this.finalAttributes,
    required this.modifiers,
    required this.hitPoints,
    required this.armorClass,
    required this.proficiencies,
    required this.savingThrows,
    required this.languages,
  });

  final AttributeSet finalAttributes;
  final Map<String, int> modifiers;
  final int hitPoints;
  final int armorClass;
  final List<String> proficiencies;
  final List<String> savingThrows;
  final List<String> languages;
}

================================================================================================================
ARQUIVO: lib/domain/entities/game_catalog.dart
================================================================================================================
import 'background.dart';
import 'character_class.dart';
import 'proficiency.dart';
import 'race.dart';

class GameCatalog {
  const GameCatalog({
    required this.races,
    required this.classes,
    required this.backgrounds,
    required this.proficiencies,
  });

  final List<Race> races;
  final List<CharacterClass> classes;
  final List<Background> backgrounds;
  final List<Proficiency> proficiencies;

  Race raceById(String id) {
    return races.firstWhere((race) => race.id == id);
  }

  CharacterClass classById(String id) {
    return classes.firstWhere((characterClass) => characterClass.id == id);
  }

  Background backgroundById(String id) {
    return backgrounds.firstWhere((background) => background.id == id);
  }
}

================================================================================================================
ARQUIVO: lib/domain/entities/proficiency.dart
================================================================================================================
class Proficiency {
  const Proficiency({required this.id, required this.name, required this.type});

  final String id;
  final String name;
  final String type;

  factory Proficiency.fromJson(Map<String, dynamic> json) {
    return Proficiency(
      id: json['id'] as String,
      name: json['name'] as String,
      type: json['type'] as String,
    );
```

```
  }
}

================================================================================================
ARQUIVO: lib/domain/entities/race.dart
================================================================================================
class Race {
  const Race({
    required this.id,
    required this.name,
    required this.bonuses,
    required this.proficiencies,
    required this.languages,
  });

  final String id;
  final String name;
  final Map<String, int> bonuses;
  final List<String> proficiencies;
  final List<String> languages;

  factory Race.fromJson(Map<String, dynamic> json) {
    return Race(
      id: json['id'] as String,
      name: json['name'] as String,
      bonuses: (json['bonuses'] as Map<String, dynamic>).map(
        (key, value) => MapEntry(key, value as int),
      ),
      proficiencies: List<String>.from(
        json['proficiencies'] as List? ?? const [],
      ),
      languages: List<String>.from(json['languages'] as List? ?? const []),
    );
  }
}

================================================================================================
ARQUIVO: lib/domain/repositories/game_catalog_repository.dart
================================================================================================
import '../entities/game_catalog.dart';

abstract class GameCatalogRepository {
  Future<GameCatalog> loadCatalog();
}

================================================================================================
ARQUIVO: lib/domain/rules/armor_class_rule.dart
================================================================================================
int calculateUnarmoredArmorClass(int dexterityModifier) {
  return 10 + dexterityModifier;
}

================================================================================================
ARQUIVO: lib/domain/rules/attribute_modifier_rule.dart
================================================================================================
int calculateAttributeModifier(int value) {
  return ((value - 10) / 2).floor();
}

================================================================================================
ARQUIVO: lib/domain/rules/hit_points_rule.dart
================================================================================================
int calculateInitialHitPoints({
  required int hitDice,
  required int constitutionModifier,
}) {
  return hitDice + constitutionModifier;
}

================================================================================================
ARQUIVO: lib/domain/rules/proficiency_rule.dart
================================================================================================
List<String> mergeProficiencies(Iterable<List<String>> groups) {
  return {
    for (final group in groups)
      for (final item in group) item,
```

```
  }.toList()..sort();
}

================================================================================================================
ARQUIVO: lib/domain/rules/racial_bonus_rule.dart
================================================================================================================
import '../entities/attribute_set.dart';
import '../entities/race.dart';

AttributeSet applyRacialBonuses(AttributeSet attributes, Race race) {
  return attributes.addBonuses(race.bonuses);
}

================================================================================================================
ARQUIVO: lib/domain/usecases/apply_race_usecase.dart
================================================================================================================
import '../entities/attribute_set.dart';
import '../entities/race.dart';
import '../rules/racial_bonus_rule.dart';

class ApplyRaceUseCase {
  const ApplyRaceUseCase();

  AttributeSet call(AttributeSet attributes, Race race) {
    return applyRacialBonuses(attributes, race);
  }
}

================================================================================================================
ARQUIVO: lib/domain/usecases/calculate_attribute_modifier_usecase.dart
================================================================================================================
import '../rules/attribute_modifier_rule.dart';

class CalculateAttributeModifierUseCase {
  const CalculateAttributeModifierUseCase();

  int call(int value) {
    return calculateAttributeModifier(value);
  }
}

================================================================================================================
ARQUIVO: lib/domain/usecases/calculate_character_stats_usecase.dart
================================================================================================================
import '../../core/constants/attribute_keys.dart';
import '../entities/character.dart';
import '../entities/character_stats.dart';
import '../entities/game_catalog.dart';
import '../rules/armor_class_rule.dart';
import '../rules/attribute_modifier_rule.dart';
import '../rules/hit_points_rule.dart';
import '../rules/proficiency_rule.dart';
import '../rules/racial_bonus_rule.dart';

class CalculateCharacterStatsUseCase {
  const CalculateCharacterStatsUseCase(this.catalog);

  final GameCatalog catalog;

  CharacterStats call(Character character) {
    final race = catalog.raceById(character.raceId);
    final characterClass = catalog.classById(character.classId);
    final background = catalog.backgroundById(character.backgroundId);
    final finalAttributes = applyRacialBonuses(character.attributes, race);
    final modifiers = {
      for (final key in attributeKeys)
        key: calculateAttributeModifier(finalAttributes.valueFor(key)),
    };
    final constitutionModifier = modifiers['constitution'] ?? 0;
    final dexterityModifier = modifiers['dexterity'] ?? 0;

    return CharacterStats(
      finalAttributes: finalAttributes,
      modifiers: modifiers,
      hitPoints: calculateInitialHitPoints(
        hitDice: characterClass.hitDice,
```

```
        constitutionModifier: constitutionModifier,
      ),
      armorClass: calculateUnarmoredArmorClass(dexterityModifier),
      proficiencies: mergeProficiencies([
        race.proficiencies,
        characterClass.proficiencies,
        background.skillProficiencies,
      ]),
      savingThrows: characterClass.savingThrows,
      languages: race.languages,
    );
  }
}
```

```
================================================================================================================
ARQUIVO: lib/domain/usecases/calculate_hitpoints_usecase.dart
================================================================================================================
import '../entities/character_class.dart';
import '../rules/hit_points_rule.dart';

class CalculateHitPointsUseCase {
  const CalculateHitPointsUseCase();

  int call(CharacterClass characterClass, int constitutionModifier) {
    return calculateInitialHitPoints(
      hitDice: characterClass.hitDice,
      constitutionModifier: constitutionModifier,
    );
  }
}
```

```
================================================================================================================
ARQUIVO: lib/domain/usecases/create_character_usecase.dart
================================================================================================================
import '../entities/character.dart';

class CreateCharacterUseCase {
  const CreateCharacterUseCase();

  Character call() {
    return const Character.initial();
  }
}
```

```
================================================================================================================
ARQUIVO: lib/main.dart
================================================================================================================
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'core/routing/app_router.dart';
import 'core/theme/app_theme.dart';

void main() {
  runApp(const ProviderScope(child: RpgSheetBuilderApp()));
}

class RpgSheetBuilderApp extends StatelessWidget {
  const RpgSheetBuilderApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp.router(
      title: 'Construtor de Ficha RPG',
      theme: buildAppTheme(),
      routerConfig: appRouter,
    );
  }
}
```

```
================================================================================================================
ARQUIVO: lib/presentation/providers/catalog_providers.dart
================================================================================================================
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../data/datasources/asset_json_datasource.dart';
```

```
import '../../data/repositories/asset_game_catalog_repository.dart';
import '../../domain/entities/game_catalog.dart';

final gameCatalogProvider = FutureProvider<GameCatalog>((ref) {
  final datasource = AssetJsonDatasource();
  final repository = AssetGameCatalogRepository(datasource);
  return repository.loadCatalog();
});
```

```
================================================================================================================
ARQUIVO: lib/presentation/providers/character_controller.dart
================================================================================================================
```

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../domain/entities/attribute_set.dart';
import '../../domain/entities/character.dart';

final characterControllerProvider =
    NotifierProvider<CharacterController, Character>(CharacterController.new);

class CharacterController extends Notifier<Character> {
  @override
  Character build() {
    return const Character.initial();
  }

  void updateName(String name) {
    state = state.copyWith(name: name);
  }

  void selectRace(String raceId) {
    state = state.copyWith(raceId: raceId);
  }

  void selectClass(String classId) {
    state = state.copyWith(classId: classId);
  }

  void selectBackground(String backgroundId) {
    state = state.copyWith(backgroundId: backgroundId);
  }

  void setAttribute(String key, int value) {
    final clamped = value.clamp(1, 20).toInt();
    state = state.copyWith(attributes: state.attributes.setValue(key, clamped));
  }

  void incrementAttribute(String key) {
    setAttribute(key, state.attributes.valueFor(key) + 1);
  }

  void decrementAttribute(String key) {
    setAttribute(key, state.attributes.valueFor(key) - 1);
  }

  void resetAttributes() {
    state = state.copyWith(attributes: const AttributeSet.standard());
  }
}
```

```
================================================================================================================
ARQUIVO: lib/presentation/providers/character_stats_provider.dart
================================================================================================================
```

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../domain/entities/character_stats.dart';
import '../../domain/usecases/calculate_character_stats_usecase.dart';
import 'catalog_providers.dart';
import 'character_controller.dart';

final characterStatsProvider = Provider<AsyncValue<CharacterStats>>((ref) {
  final catalog = ref.watch(gameCatalogProvider);
  final character = ref.watch(characterControllerProvider);

  return catalog.whenData((value) {
    return CalculateCharacterStatsUseCase(value).call(character);
```

```
  });
});

================================================================================================
ARQUIVO: lib/presentation/screens/character_builder_screen.dart
================================================================================================
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../providers/catalog_providers.dart';
import '../providers/character_controller.dart';
import '../widgets/attribute_editor.dart';
import '../widgets/results_panel.dart';

class CharacterBuilderScreen extends ConsumerWidget {
  const CharacterBuilderScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final catalog = ref.watch(gameCatalogProvider);

    return Scaffold(
      appBar: AppBar(title: const Text('Construtor de Ficha RPG - V1')),
      body: catalog.when(
        data: (_) => const _CharacterBuilderContent(),
        error: (error, stackTrace) =>
            Center(child: Text('Erro ao carregar dados locais: $error')),
        loading: () => const Center(child: CircularProgressIndicator()),
      ),
    );
  }
}

class _CharacterBuilderContent extends ConsumerWidget {
  const _CharacterBuilderContent();

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final catalog = ref.watch(gameCatalogProvider).requireValue;
    final character = ref.watch(characterControllerProvider);
    final controller = ref.read(characterControllerProvider.notifier);

    return LayoutBuilder(
      builder: (context, constraints) {
        final controls = _SelectionCard(
          raceId: character.raceId,
          classId: character.classId,
          backgroundId: character.backgroundId,
          onNameChanged: controller.updateName,
          onRaceChanged: controller.selectRace,
          onClassChanged: controller.selectClass,
          onBackgroundChanged: controller.selectBackground,
          races: catalog.races
              .map((race) => (id: race.id, name: race.name))
              .toList(),
          classes: catalog.classes
              .map(
                (characterClass) =>
                    (id: characterClass.id, name: characterClass.name),
              )
              .toList(),
          backgrounds: catalog.backgrounds
              .map((background) => (id: background.id, name: background.name))
              .toList(),
        );
        const attributes = AttributeEditor();
        const results = ResultsPanel();

        final content = constraints.maxWidth >= 920
            ? Row(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Expanded(
                    child: Column(
                      children: [
                        controls,
```

```
                      const SizedBox(height: 16),
                      attributes,
                    ],
                  ),
                ),
                const SizedBox(width: 16),
                const Expanded(child: results),
              ],
            )
          : Column(
              children: [
                controls,
                const SizedBox(height: 16),
                attributes,
                const SizedBox(height: 16),
                results,
              ],
            );

      return SingleChildScrollView(
        padding: const EdgeInsets.all(16),
        child: content,
      );
    },
  );
 }
}

class _SelectionCard extends StatelessWidget {
  const _SelectionCard({
    required this.raceId,
    required this.classId,
    required this.backgroundId,
    required this.onNameChanged,
    required this.onRaceChanged,
    required this.onClassChanged,
    required this.onBackgroundChanged,
    required this.races,
    required this.classes,
    required this.backgrounds,
  });

  final String raceId;
  final String classId;
  final String backgroundId;
  final ValueChanged<String> onNameChanged;
  final ValueChanged<String> onRaceChanged;
  final ValueChanged<String> onClassChanged;
  final ValueChanged<String> onBackgroundChanged;
  final List<({String id, String name})> races;
  final List<({String id, String name})> classes;
  final List<({String id, String name})> backgrounds;

  @override
  Widget build(BuildContext context) {
    return Card(
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              'Criacao de personagem',
              style: Theme.of(context).textTheme.titleLarge,
            ),
            const SizedBox(height: 12),
            TextFormField(
              decoration: const InputDecoration(
                labelText: 'Nome',
                border: OutlineInputBorder(),
              ),
              onChanged: onNameChanged,
            ),
            const SizedBox(height: 12),
            _SelectionDropdown(
              label: 'Raca',
```

```
              value: raceId,
              options: races,
              onChanged: onRaceChanged,
            ),
            const SizedBox(height: 12),
            _SelectionDropdown(
              label: 'Classe',
              value: classId,
              options: classes,
              onChanged: onClassChanged,
            ),
            const SizedBox(height: 12),
            _SelectionDropdown(
              label: 'Antecedente',
              value: backgroundId,
              options: backgrounds,
              onChanged: onBackgroundChanged,
            ),
          ],
        ),
      ),
    );
  }
}

class _SelectionDropdown extends StatelessWidget {
  const _SelectionDropdown({
    required this.label,
    required this.value,
    required this.options,
    required this.onChanged,
  });

  final String label;
  final String value;
  final List<({String id, String name})> options;
  final ValueChanged<String> onChanged;

  @override
  Widget build(BuildContext context) {
    return DropdownButtonFormField<String>(
      initialValue: value,
      decoration: InputDecoration(
        labelText: label,
        border: const OutlineInputBorder(),
      ),
      items: [
        for (final option in options)
          DropdownMenuItem(value: option.id, child: Text(option.name)),
      ],
      onChanged: (value) {
        if (value != null) {
          onChanged(value);
        }
      },
    );
  }
}

================================================================================================
ARQUIVO: lib/presentation/widgets/attribute_editor.dart
================================================================================================
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../core/constants/attribute_keys.dart';
import '../providers/character_controller.dart';
import '../providers/character_stats_provider.dart';

class AttributeEditor extends ConsumerWidget {
  const AttributeEditor({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final character = ref.watch(characterControllerProvider);
    final stats = ref.watch(characterStatsProvider);
```

```
    final currentStats = switch (stats) {
      AsyncData(:final value) => value,
      _ => null,
    };
    final controller = ref.read(characterControllerProvider.notifier);

    return Card(
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Row(
              children: [
                Text(
                  'Atributos',
                  style: Theme.of(context).textTheme.titleLarge,
                ),
                const Spacer(),
                TextButton(
                  onPressed: controller.resetAttributes,
                  child: const Text('Resetar'),
                ),
              ],
            ),
            const SizedBox(height: 8),
            for (final key in attributeKeys)
              _AttributeRow(
                label: attributeLabels[key] ?? key,
                baseValue: character.attributes.valueFor(key),
                finalValue: currentStats?.finalAttributes.valueFor(key),
                modifier: currentStats?.modifiers[key],
                onDecrease: () => controller.decrementAttribute(key),
                onIncrease: () => controller.incrementAttribute(key),
              ),
          ],
        ),
      ),
    );
  }
}

class _AttributeRow extends StatelessWidget {
  const _AttributeRow({
    required this.label,
    required this.baseValue,
    required this.finalValue,
    required this.modifier,
    required this.onDecrease,
    required this.onIncrease,
  });

  final String label;
  final int baseValue;
  final int? finalValue;
  final int? modifier;
  final VoidCallback onDecrease;
  final VoidCallback onIncrease;

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);

    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 6),
      child: Row(
        children: [
          Expanded(child: Text(label, style: theme.textTheme.titleSmall)),
          IconButton.filledTonal(
            onPressed: onDecrease,
            icon: const Icon(Icons.remove),
          ),
          SizedBox(
            width: 42,
            child: Text(
              '$baseValue',
```

```
              textAlign: TextAlign.center,
              style: theme.textTheme.titleMedium,
            ),
          ),
          IconButton.filledTonal(
            onPressed: onIncrease,
            icon: const Icon(Icons.add),
          ),
          const SizedBox(width: 16),
          SizedBox(
            width: 94,
            child: Text(
              'Final: ${finalValue ?? '-'}',
              style: theme.textTheme.bodyMedium,
            ),
          ),
          SizedBox(
            width: 70,
            child: Text(
              'Mod: ${modifier == null ? '-' : _formatSigned(modifier!)}',
              style: theme.textTheme.bodyMedium,
            ),
          ),
        ],
      ),
    );
  }
}

String _formatSigned(int value) {
  return value >= 0 ? '+$value' : '$value';
}

================================================================================================================
ARQUIVO: lib/presentation/widgets/results_panel.dart
================================================================================================================
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../core/constants/attribute_keys.dart';
import '../providers/character_stats_provider.dart';

class ResultsPanel extends ConsumerWidget {
  const ResultsPanel({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final stats = ref.watch(characterStatsProvider);

    return Card(
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: stats.when(
          data: (value) => Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(
                'Resultados em tempo real',
                style: Theme.of(context).textTheme.titleLarge,
              ),
              const SizedBox(height: 12),
              Wrap(
                spacing: 12,
                runSpacing: 12,
                children: [
                  _StatChip(label: 'PV inicial', value: '${value.hitPoints}'),
                  _StatChip(
                    label: 'CA sem armadura',
                    value: '${value.armorClass}',
                  ),
                  _StatChip(
                    label: 'Testes de resistencia',
                    value: _formatList(value.savingThrows),
                  ),
                  _StatChip(
                    label: 'Idiomas',
```

```
                    value: _formatList(value.languages),
                  ),
                ],
              ),
              const Divider(height: 28),
              Text(
                'Modificadores',
                style: Theme.of(context).textTheme.titleMedium,
              ),
              const SizedBox(height: 8),
              Wrap(
                spacing: 8,
                runSpacing: 8,
                children: [
                  for (final key in attributeKeys)
                    Chip(
                      label: Text(
                        '${attributeLabels[key]} ${_formatSigned(value.modifiers[key] ?? 0)}',
                      ),
                    ),
                ],
              ),
              const Divider(height: 28),
              Text(
                'Proficiencias',
                style: Theme.of(context).textTheme.titleMedium,
              ),
              const SizedBox(height: 8),
              Text(_formatList(value.proficiencies)),
            ],
          ),
          error: (error, stackTrace) => Text('Erro ao calcular ficha: $error'),
          loading: () => const Center(child: CircularProgressIndicator()),
        ),
      ),
    );
  }
}

class _StatChip extends StatelessWidget {
  const _StatChip({required this.label, required this.value});

  final String label;
  final String value;

  @override
  Widget build(BuildContext context) {
    return Chip(
      label: Text('$label: $value'),
      padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 6),
    );
  }
}

String _formatList(List<String> values) {
  if (values.isEmpty) {
    return '-';
  }

  return values.map(_humanize).join(', ');
}

String _humanize(String value) {
  final translated = _displayNames[value];
  if (translated != null) {
    return translated;
  }

  return value
      .split('_')
      .map(
        (part) => part.isEmpty
            ? part
            : '${part[0].toUpperCase()}${part.substring(1)}',
      )
      .join(' ');
```

```
}

String _formatSigned(int value) {
  return value >= 0 ? '+$value' : '$value';
}

const _displayNames = <String, String>{
  'strength': 'Forca',
  'dexterity': 'Destreza',
  'constitution': 'Constituicao',
  'intelligence': 'Inteligencia',
  'wisdom': 'Sabedoria',
  'charisma': 'Carisma',
  'perception': 'Percepcao',
  'battleaxe': 'Machado de batalha',
  'handaxe': 'Machadinha',
  'light_hammer': 'Martelo leve',
  'warhammer': 'Martelo de guerra',
  'all_armor': 'Todas as armaduras',
  'shields': 'Escudos',
  'simple_weapons': 'Armas simples',
  'martial_weapons': 'Armas marciais',
  'dagger': 'Adaga',
  'dart': 'Dardo',
  'sling': 'Funda',
  'quarterstaff': 'Bordao',
  'light_crossbow': 'Besta leve',
  'light_armor': 'Armadura leve',
  'medium_armor': 'Armadura media',
  'hand_crossbow': 'Besta de mao',
  'longsword': 'Espada longa',
  'rapier': 'Rapieira',
  'shortsword': 'Espada curta',
  'thieves_tools': 'Ferramentas de ladrao',
  'insight': 'Intuicao',
  'religion': 'Religiao',
  'athletics': 'Atletismo',
  'intimidation': 'Intimidacao',
  'deception': 'Enganacao',
  'stealth': 'Furtividade',
  'arcana': 'Arcanismo',
  'history': 'Historia',
};

================================================================================================
ARQUIVO: assets/data/backgrounds.json
================================================================================================
[
  {
    "id": "acolyte",
    "name": "Acolito",
    "skillProficiencies": [
      "insight",
      "religion"
    ]
  },
  {
    "id": "soldier",
    "name": "Soldado",
    "skillProficiencies": [
      "athletics",
      "intimidation"
    ]
  },
  {
    "id": "criminal",
    "name": "Criminoso",
    "skillProficiencies": [
      "deception",
      "stealth"
    ]
  },
  {
    "id": "sage",
    "name": "Sabio",
    "skillProficiencies": [
```

```
      "arcana",
      "history"
    ]
  }
]
```

```
================================================================================================================
ARQUIVO: assets/data/classes.json
================================================================================================================
```

```
[
  {
    "id": "fighter",
    "name": "Guerreiro",
    "hitDice": 10,
    "savingThrows": [
      "strength",
      "constitution"
    ],
    "proficiencies": [
      "all_armor",
      "shields",
      "simple_weapons",
      "martial_weapons"
    ]
  },
  {
    "id": "wizard",
    "name": "Mago",
    "hitDice": 6,
    "savingThrows": [
      "intelligence",
      "wisdom"
    ],
    "proficiencies": [
      "dagger",
      "dart",
      "sling",
      "quarterstaff",
      "light_crossbow"
    ]
  },
  {
    "id": "rogue",
    "name": "Ladino",
    "hitDice": 8,
    "savingThrows": [
      "dexterity",
      "intelligence"
    ],
    "proficiencies": [
      "light_armor",
      "simple_weapons",
      "hand_crossbow",
      "longsword",
      "rapier",
      "shortsword",
      "thieves_tools"
    ]
  },
  {
    "id": "cleric",
    "name": "Clerigo",
    "hitDice": 8,
    "savingThrows": [
      "wisdom",
      "charisma"
    ],
    "proficiencies": [
      "light_armor",
      "medium_armor",
      "shields",
      "simple_weapons"
    ]
  }
]
```

```
================================================================================================================
ARQUIVO: assets/data/proficiencies.json
================================================================================================================
[
  {
    "id": "arcana",
    "name": "Arcanismo",
    "type": "skill"
  },
  {
    "id": "history",
    "name": "Historia",
    "type": "skill"
  },
  {
    "id": "insight",
    "name": "Intuicao",
    "type": "skill"
  },
  {
    "id": "religion",
    "name": "Religiao",
    "type": "skill"
  },
  {
    "id": "athletics",
    "name": "Atletismo",
    "type": "skill"
  },
  {
    "id": "intimidation",
    "name": "Intimidacao",
    "type": "skill"
  },
  {
    "id": "deception",
    "name": "Enganacao",
    "type": "skill"
  },
  {
    "id": "stealth",
    "name": "Furtividade",
    "type": "skill"
  },
  {
    "id": "perception",
    "name": "Percepcao",
    "type": "skill"
  }
]

================================================================================================================
ARQUIVO: assets/data/races.json
================================================================================================================
[
  {
    "id": "elf",
    "name": "Elfo",
    "bonuses": {
      "dexterity": 2
    },
    "proficiencies": [
      "perception"
    ],
    "languages": [
      "Comum",
      "Elfico"
    ]
  },
  {
    "id": "human",
    "name": "Humano",
    "bonuses": {
      "strength": 1,
      "dexterity": 1,
      "constitution": 1,
```

```
      "intelligence": 1,
      "wisdom": 1,
      "charisma": 1
    },
    "proficiencies": [],
    "languages": [
      "Comum"
    ]
  },
  {
    "id": "dwarf",
    "name": "Anao",
    "bonuses": {
      "constitution": 2
    },
    "proficiencies": [
      "battleaxe",
      "handaxe",
      "light_hammer",
      "warhammer"
    ],
    "languages": [
      "Comum",
      "Anao"
    ]
  },
  {
    "id": "halfling",
    "name": "Pequenino",
    "bonuses": {
      "dexterity": 2
    },
    "proficiencies": [],
    "languages": [
      "Comum",
      "Pequenino"
    ]
  }
]

================================================================================================
ARQUIVO: test/domain/rules_test.dart
================================================================================================
import 'package:flutter_test/flutter_test.dart';
import 'package:rpg_sheet_builder/domain/entities/background.dart';
import 'package:rpg_sheet_builder/domain/entities/character.dart';
import 'package:rpg_sheet_builder/domain/entities/character_class.dart';
import 'package:rpg_sheet_builder/domain/entities/game_catalog.dart';
import 'package:rpg_sheet_builder/domain/entities/race.dart';
import 'package:rpg_sheet_builder/domain/rules/attribute_modifier_rule.dart';
import 'package:rpg_sheet_builder/domain/usecases/calculate_character_stats_usecase.dart';

void main() {
  group('Modificadores de atributo', () {
    test('seguem a formula de D&D 5e', () {
      expect(calculateAttributeModifier(10), 0);
      expect(calculateAttributeModifier(8), -1);
      expect(calculateAttributeModifier(15), 2);
      expect(calculateAttributeModifier(20), 5);
    });
  });

  group('Estatisticas do personagem', () {
    test('Elfo aplica +2 Destreza e Guerreiro tem PV 10 + CON', () {
      final stats = const CalculateCharacterStatsUseCase(
        _catalog,
      ).call(const Character.initial());

      expect(stats.finalAttributes.dexterity, 12);
      expect(stats.modifiers['dexterity'], 1);
      expect(stats.hitPoints, 10);
      expect(stats.proficiencies, contains('athletics'));
      expect(stats.proficiencies, contains('simple_weapons'));
    });

    test('Humano aplica +1 em todos os atributos', () {
```

```
      final stats = const CalculateCharacterStatsUseCase(
        _catalog,
      ).call(const Character.initial().copyWith(raceId: 'human'));

      expect(stats.finalAttributes.strength, 11);
      expect(stats.finalAttributes.dexterity, 11);
      expect(stats.finalAttributes.constitution, 11);
      expect(stats.finalAttributes.intelligence, 11);
      expect(stats.finalAttributes.wisdom, 11);
      expect(stats.finalAttributes.charisma, 11);
    });

    test('Mago tem PV 6 + CON e Sabio aplica Arcanismo + Historia', () {
      final stats = const CalculateCharacterStatsUseCase(_catalog).call(
        const Character.initial().copyWith(
          classId: 'wizard',
          backgroundId: 'sage',
        ),
      );

      expect(stats.hitPoints, 6);
      expect(stats.savingThrows, containsAll(['intelligence', 'wisdom']));
      expect(stats.proficiencies, contains('arcana'));
      expect(stats.proficiencies, contains('history'));
    });
  });
}

const _catalog = GameCatalog(
  races: [
    Race(
      id: 'elf',
      name: 'Elfo',
      bonuses: {'dexterity': 2},
      proficiencies: ['perception'],
      languages: ['Comum', 'Elfico'],
    ),
    Race(
      id: 'human',
      name: 'Humano',
      bonuses: {
        'strength': 1,
        'dexterity': 1,
        'constitution': 1,
        'intelligence': 1,
        'wisdom': 1,
        'charisma': 1,
      },
      proficiencies: [],
      languages: ['Comum'],
    ),
  ],
  classes: [
    CharacterClass(
      id: 'fighter',
      name: 'Guerreiro',
      hitDice: 10,
      savingThrows: ['strength', 'constitution'],
      proficiencies: ['simple_weapons', 'martial_weapons'],
    ),
    CharacterClass(
      id: 'wizard',
      name: 'Mago',
      hitDice: 6,
      savingThrows: ['intelligence', 'wisdom'],
      proficiencies: ['dagger', 'quarterstaff'],
    ),
  ],
  backgrounds: [
    Background(
      id: 'soldier',
      name: 'Soldado',
      skillProficiencies: ['athletics', 'intimidation'],
    ),
    Background(
      id: 'sage',
```

```
      name: 'Sabio',
      skillProficiencies: ['arcana', 'history'],
    ),
  ],
  proficiencies: [],
);

================================================================================================================
ARQUIVO: test/widget_test.dart
================================================================================================================
import 'package:flutter_test/flutter_test.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:rpg_sheet_builder/main.dart';

void main() {
  testWidgets('exibe o construtor de personagem da V1', (tester) async {
    await tester.pumpWidget(const ProviderScope(child: RpgSheetBuilderApp()));
    await tester.pumpAndSettle();

    expect(find.text('Construtor de Ficha RPG - V1'), findsOneWidget);
    expect(find.text('Criacao de personagem'), findsOneWidget);
    expect(find.text('Resultados em tempo real'), findsOneWidget);
    expect(find.textContaining('PV inicial'), findsOneWidget);
  });
}
```